

Make assert() macro user friendly for C and C++

Document #: P2246R0 (WG21)/ N2621 (WG14)
Date: 2020-12-16
Project: Programming Language C++
Audience: WG21 - Library Evolution and SG22 (C++/C-Liaison), WG14
Reply-to: Peter Sommerlad
<peter.cpp@sommerlad.ch>

Contents

- [1 Introduction](#)
- [2 Remedy](#)
- [3 Impact on existing code](#)
- [4 Potential liabilities of the proposed change](#)
- [5 Wording for C++](#)
 - [5.0.1 19.3.2 The assert macro \[assertions.assert\]](#)
- [6 Wording for C](#)
 - [6.0.1 7.2.1 Program diagnostics](#)
- [7 Acknowledgements](#)

§ 1 Introduction

The `assert()` macro, being a macro, is not very beginner friendly in C++, because the preprocessor only uses parenthesis for pairing and none of the other structuring syntax of C++, such as template angle brackets, or curly braces. This makes it user unfriendly in a C++ context, requiring an extra pair of parentheses, if the expression used, incorporates a comma.

Shafik Yaghmour presented the following C++ code in one of his Twitter quizzes [tweet](#) demonstrating the weakness.

```
#include <cassert>
#include <type_traits>

using Int=int;
```

```
void f() {
    assert(std::is_same<int, Int>::value); // a surprisig compile error
}
```

One of the twitter [responses](#) (by [@_Static_assert](#)) to the tweet mentioned above, even provided a definition of the assert macro that actually is a primitive implementation of what I propose in this paper:

```
#define assert(...) ((__VA_ARGS__)?(void)0:std::abort())
```

In C one needs to be a bit more sophisticated to trigger such a compile error, but nevertheless the C syntax allows for such expression that include commas that are not protected from the preprocessor by parantheses as given by Shafik's [godbolt example](#)

```
#include <assert.h>

void f() {
    assert((int[2]){1,2}[0]); // compile error
    struct A {int x,y;};
    assert((struct A){1,2}.x); // compile error
}
```

The current C standard does not even sanction such a compile error to my knowledge, when NDEBUG is not defined, since it specifies the assert macro to be able to take an expression of *scalar type* which the above non-compiling examples with a comma, I think, are (int in both cases). The C++ standard and working paper refer to C's definition of the assert macro in that respect.

§ 2 Remedy

This deficit in the single argument macro assert() seems to be very easy to mitigate by providing a __VA_ARGS__ version of the macro using ellipsis (...) as the parameter.

There exist the option to specify the assert macro with an extra name parameter and then the ellipsis. However, I do think this not only complicates its implementation it also complicates its wording. If the assert macro is called without any arguments this will lead to a compile error as it does today. The only difference might be the issued compiler diagnostic

A DIY version can be defined that provides the additional parenthesis needed for the assert() macro of today:

```
#define Assert(...) assert((__VA_ARGS__))
```

However, that would be required to be defined and used throughout a project and such is less user friendly than have the standard facility provide such flexibility.

In addition the variable argument macro version of assert would allow additional diagnostics text, by using the comma operator, such as in

```
assert((void)"this cannot be true", -0.0);
```

which would otherwise also be required to use an extra pair of parentheses.

However, such additional diagnostic strings are better spelled using the `&&` conjugation (thanks to Martin Hořeňovský)

```
assert(idx < vec.size() && "idx is out of range");
```

§ 3 Impact on existing code

I could not do deep analysis of large code bases but as best to my knowledge the suggestion should not break any existing code but will make code successfully compile, that previously ended up in a compile error, due to the limitations of `assert()` arguments with commas in them.

When reading the C specification of the semantics of `assert()` one could argue that the macro parameter should already have been variadic, because even in C one can form a scalar expression with a comma that doesn't require balanced parentheses. So WG14 and WG21 might even consider to apply this change as a backward defect fix to previous revisions of the standards.

§ 4 Potential liabilities of the proposed change

While sharing a preview of this document I got several people commenting on it. While some were in favor, there were raised some potential issue that I'd like to share paraphrased below:

1. “Contracts will make the 50 year old `assert` macro obsolete and to not suffer from the macro parsing issue.”
2. “Using the comma operator can be misapplied to an always true `assert`, if its arguments are formed as for `static_assert(condition, reason)`. This will make wrong code compile that today doesn't.”
3. “Teachability is not improved, because we can teach use extra parentheses today.”

Nevertheless, I think it is worthwhile to make `assert()` more beginner friendly, since professional code bases will have their own versions of precondition checking stuff anyway. While beginners can be shown a universally available feature that is identical to C.

There were liabilities that I do not list above, because they are already addressed by this paper

— breaking compatibility with C: This is not planned, see this paper.

- wording only addresses the NDEBUG case: I provide now more context, the wording changes required definitely only relevant with the NDEBUG case.

§ 5 Wording for C++

The change is relative to n4861.

In [cassert.syn] change the macro definition as follows:

```
#define assert( E ... ) see below
```

The contents are the same as the C standard library header `<assert.h>`, except that a macro named `static_assert` is not defined.

See also: ISO C 7.2

In [assertions.assert] no change is required. It is provided here for easier reference by reviewers.

§ 5.0.1 19.3.2 The `assert` macro [assertions.assert]

- 1 An expression `assert(E)` is a constant subexpression (16.3.6), if
 - `NDEBUG` is defined at the point where `assert` is last defined or redefined, or
 - `E` contextually converted to `bool` (7.3) is a constant subexpression that evaluates to the value `true`.

§ 6 Wording for C

These changes are relative to N2478.

In section 7.2 (Diagnostics <assert.h>) change the definition of the `assert()` macro to use ellipsis instead of a single macro parameter:

- 1 The header `<assert.h>` defines the `assert` and `static_assert` macros and refers to another macro,

```
NDEBUG
```

which is not defined by `<assert.h>`. If `NDEBUG` is defined as a macro name at the point in the source file where `<assert.h>` is included, the `assert` macro is defined simply as

```
- #define assert(ignore) ((void)0)
+ #define assert(...) ((void)0)
```

The `assert` macro is redefined according to the current state of `NDEBUG` each time that `<assert.h>` is included.

- 2 The `assert` macro shall be implemented as a macro with an ellipsis parameter, not as an actual function. If the macro definition is suppressed in order to access an actual function, the behavior is undefined.

In section 7.2.1 (Program Diagnostics) no change is needed. It is included here for easier reference by reviewers.

§ 6.0.1 7.2.1 Program diagnostics

§ 6.0.1.1 7.2.1.1 The `assert` macro

Synopsis

```
1 #include <assert.h>
   void assert(scalar expression);
```

Description

- 2 The `assert` macro puts diagnostic tests into programs; it expands to a void expression. When it is executed, if `expression` (which shall have a scalar type) is false (that is, compares equal to 0), the `assert` macro writes information about the particular call that failed (including the text of the argument, the name of the source file, the source line number, and the name of the enclosing function – the latter are respectively the values of the preprocessing macros `__FILE__` and `__LINE__` and of the identifier `__func__`) on the standard error stream in an implementation-defined format.¹ It then calls the `abort` function.

Returns

- 3 The `assert` macro returns no value.

Forward references: the `abort` function (7.22.4.1).

§ 7 Acknowledgements

Many thanks to Shafik Yaghmour and other Twitterers for inspiring this “janitorial” clean up paper.

1. The message might be of the form: Assertion failed: `_expression_` function `_abc_`, file `_xyz_` line `_nnn_`.↩